The one queue

Seven waves, forty-five items, six built. This document is not an eighth wave. It is the consolidation of the other seven: every live item in one list, the duplicates merged and named, the dead ones buried with a reason, and an order that can be argued with once instead of seven times.

Read this first. The wave documents are history and evidence; this is the queue.

1. The count

Measure	Value
Items written across seven waves	45
Merged away as duplicates	13
Distinct items	32
Built	6
Live in the queue	19
Buried with a reason	7
Queue items that need no build at all	4 of 19

Forty-five became thirty-two because the same ideas were re-proposed under new names as the context changed: the first hour of a new member appears in four waves, the identity kit in four, search in three, the two fixes in three. That repetition is the strongest argument for this page existing.

Six built, nineteen queued and seven buried: the sum is the thirty-two distinct items, and each one has exactly one state.

2. The queue

Every row names its sources so a reader can go back to the evidence; several rows are the merge of two or three sections written months apart in planning time.

#	Item	Sources	Kind	Needs	Proven by
Q1	An update offer: the new worker is active, the page is stale	w3 §3.6, w5 §3.1	defect	app	two builds deployed; an open tab shows a notice and one reload lands the new build id
Q2	Offline page, and a queue for what was typed	w5 §3.2	defect	app	airplane mode shows our page; a typed message sends on reconnect with the real send time
Q3	Search that is true: backfill on first use, say when it is filling	w2 §3.2, w5 §3.4	defect	api, app	first search indexes nine channels; a cold index never renders as "no results"

#	Item	Sources	Kind	Needs	Proven by
Q4	The first hour: one welcome, and a handbook that is pinned	w1 §3.5, w3 §3.5, w5 §3.3, w7 §2.4	feature	api, config	a fresh account receives exactly one message; three items are pinned in one channel
Q5	Identity kit: invite splash, banner, mascot emoji pack	w1 §3.6, w3 §3.3, w5 §3.5, w7 §2.3	content	none	three fields non-null; the invite preview shows them as a guest would see them
Q6	Mail that matches the room	w3 §3.4, w5 §3.6	content	api	a reset email shows the face, the wordmark, a plain-text part, and the correct host
Q7	An invite that reads like an invitation	w7 §2.1	feature	app, api	an invite can be created, previewed, expires, and names who sent it
Q8	Operator panel: is it alive	w1 §3.4, w5 §3.7	feature	app, api	the page shows build id, last backup age, index document counts; no secret appears in it
Q9	A moderation decision that carries a reason	w5 §3.8	feature	api	a report cannot close without a reason, and the subject can read it
Q10	Quiet defaults: mentions-only, no typing echo	w6 §2.1	feature	api, app	a new member arrives mentions-only; an existing member's choice is untouched
Q11	Catch-up: what happened while you were away	w6 §2.2	feature	app	after 24h the panel appears, its counts match the database, and nothing is marked read
Q12	Hold until morning	w6 §2.3	feature	api, app	a held message is cancellable, sends at the chosen time, and carries the time it actually sent
Q13	Letters: a slower composer	w6 §2.4	feature	app	the recipient sees a letter as a letter; no length limit is raised
Q14	Keepsakes: collections that export to markdown	w6 §2.5	feature	app	a collection exports to a file that opens with no styling and no network
Q15	Threads, in the slow channels first	w7 §2.2	config, UX	app	a thread exists, archives itself, and notifies only the people in it
Q16	A weekly hour, as a scheduled event	w7 §2.5	config	none	the event exists and appears in the client
Q17	One webhook, so the place is useful on a quiet day	w7 §2.6	config	none	a feed channel receives a post without anyone typing, and can be muted as a unit
Q18	Polls, for the person who will not type	w7 §2.7	config	none	one poll is run, answered and closed
Q19	A front door for strangers	w7 §2.8	page	proxy	`/welcome` returns 200 with JavaScript off, carries no sign-up form, and names nobody's address

3. The next three, with pass criteria

Not the best three. The three that are worth doing before anything else, in order, each with the number that decides whether it worked.

Q1 and Q2 together — one app-proxy build, one deploy, one revert. They live in the same file tree and they finish wave three's §3.6, the section that started all of this. Pass: deploy twice; the first tab shows the notice without being forced; one reload lands the second build id; with the network cut, the page still says something ours and a typed message survives the reload. **Fails if:** the notice appears while someone is typing, or an offline draft is shown as sent.

Q3 — the index tells the truth. Pass: search a cold channel and watch it fill with a count; then confirm the next message indexes without a search. **Fails if:** an empty result and an empty index look the same on screen.

Q4 — the first hour. Pass: a fresh test account receives exactly one message, on one line, with no exclamation marks, and finds three pinned items in one channel. **Fails if:** it arrives twice, or a second join sends a second welcome.

After those three, Q5 costs nothing and can run in parallel with anything: it is three uploads and a drawing of the mascot twelve times.

4. Buried, and why

Item	Where	Why it is not in the queue
Restart gate	w1 §3.3	Folded into the deploy gate; `rexctl restart` plus the health check already is the gate
Dormant rebrand files	w4 §4.4	Settled by shipping two images; the remaining files are inert until something needs the app or admin source
CI that builds	w4 §4.5	Decided: the server builds, the workflows stay unauthorised. Revisit only if a second machine builds
Desktop build claim	w2 §3.6	No release exists and none is planned; the claim is struck rather than planned
A call actually made	w2 §3.5	Needs two devices and one person; it is a test, not a feature
Push that reaches a phone	w2 §3.3	The same — a device test with no code behind it
Read receipts	w6 §4	The absence is the feature; recorded in `BRAND.md` as a stance, not as work
Second search backend, paid tiers, federation, mobile app	w5 §5, w7 §4	Rejected with reasons in the waves; not reopened here

5. Retirements

Document	Status	Still worth reading for
`PLAN.md`	history	the six operator questions and the backup argument

Document	Status	Still worth reading for
`PLAN-WAVE-2.md`	history	the measurement method and the two findings it produced
`PLAN-WAVE-3.md`	history	the status board and §3.6's evidence table
`PLAN-WAVE-4.md`	history	how a build is made, deployed and reverted — the runbook lives here
`PLAN-WAVE-5.md`	history	the eight questions, and why three of them came first
`PLAN-WAVE-6.md`	history	the quiet stance, which has moved into `BRAND.md`
`PLAN-WAVE-7.md`	history	the capability inventory: nine features, no rows
`BACKLOG.md`	**live**	**the queue, the order, the pass criteria**
`IDEAS.md`	parked	eight new features with falsifiers; nothing there enters the queue without displacing something here

6. When to write another document

One condition, and it is not "the owner asked". Write another plan only when **a measurement contradicts this queue** — a row that turns out to be already built, a defect that is worse than described, or a new number that changes the order of §3.

What does not justify another document: a new idea, because new ideas go in the rejected table with a reason; a re-reading of an old wave, because the retirement table already says what each is for; or the fact that planning is easier than building.

The queue is nineteen items deep and six have been built. Nineteen is not a backlog problem, it is a Tuesday. The next action is Q1 and Q2, and it is code.